Studijní materiál: Rozhodování agentů v multiagentních systémech (MAS) a Teorie her

Motivační shrnutí

Vítejte v tomto studijním modulu! V moderní umělé inteligenci se málokdy setkáváme se systémy, které působí zcela izolovaně. Často operují v prostředí, kde působí další autonomní subjekty – jiní agenti. Jak mohou agenti efektivně rozhodovat, plánovat a předvídat kroky ostatních? Tento studijní materiál vás provede základními koncepty rozhodování v multiagentních systémech (MAS) a matematickým aparátem teorie her, včetně strategických konceptů jako Nashova rovnováha či Paretovo optimum. Následně se zaměříme na deterministické hry s úplnou informací, vyhledávací algoritmus Minimax, jeho optimalizaci pomocí alfa-beta ořezávání a řešení reálných problémů za použití heuristických evaluačních funkcí a hloubkových limitů.

Souvislý studijní materiál (Distanční vzdělávání)

Kapitola 1: Rozhodování agentů v MAS

Výukové cíle kapitoly:

- Porozumět rozdílům mezi strategií a plánem agenta.
- Definovat ryzí, smíšenou a dominantní strategii.
- Vysvětlit koncept Nashovy rovnováhy a Paretova optima.

1.1 Základní koncepty rozhodování

Multiagentní systém (MAS) představuje prostředí složené z více interagujících autonomních agentů. Pro správné rozhodování agenta rozlišujeme následující pojmy:

- **Strategie agenta**: Koncept, který určuje způsob, jakým agent rozhoduje a jedná ve vztahu k ostatním agentům. Může být definována jako závislost akce na stavu nebo vývoji prostředí či chování ostatních agentů.
- **Plán agenta**: Konkrétní sekvence akcí, které má agent provést, aby dosáhl svého cíle. Během provádění může být na základě aktuálních podmínek dynamicky aktualizován.

1.2 Typy strategií

- **Ryzí strategie**: Konkrétní a jednoznačný (deterministický) plán, který určuje, jaké akce agent provede v každé možné situaci (např. ve hře Kámen, nůžky, papír agent vždy zvolí kámen).
- **Příklad z Product Managementu (Strategie vs. Plán):**
    - *Produktová strategie:* „Soustředíme se na enterprise zákazníky zvýšením důrazu na bezpečnost a pravidla plnění compliance.“ (Určuje reakce na změny trhu a chování konkurence).
    - *Release plán (Roadmapa):* „Sprint 1: SSO/SAML, Sprint 2: Audit logy, Sprint 3: SOC2 certifikace.“ (Sekvence konkrétních vývojových kroků).
- **Smíšená strategie**: Agent má několik ryzích strategií a volí mezi nimi s určitým pravděpodobnostním rozložením (např. volí kámen, nůžky i papír s pravděpodobností 1/3).
- **Dominantní strategie**: Strategie, která přináší agentu nejlepší možný výsledek bez ohledu na to, jakou strategii zvolí ostatní agenti. Může být ryzí i smíšená. Pokud existuje dominantní strategie, racionální agent volí právě tuto strategii s cílem maximalizovat svůj užitek.
- **Příklad v Software Developmentu (Typy strategií):**
    - *Ryzí strategie:* Každý nový mikroslužební endpoint striktně zakrýt feature flagem.
    - *Smíšená strategie (Canary deployment):* Směrovat 90 % provozu na stabilní verzi v2.1 a 10 % provozu na kandidátskou verzi v2.2.
    - *Dominantní strategie:* Implementace CI/CD pipeline s automatickými testy a bezpečnostním skenem před nasazením na produkci – přináší nejlepší výsledek v kvalitě bez ohledu na kroky konkurence.

1.3 Nashova rovnováha

Nashova rovnováha popisuje stav, kdy žádný agent nemůže zlepšit svůj užitek jednostrannou změnou své strategie za předpokladu, že ostatní agenti své strategie

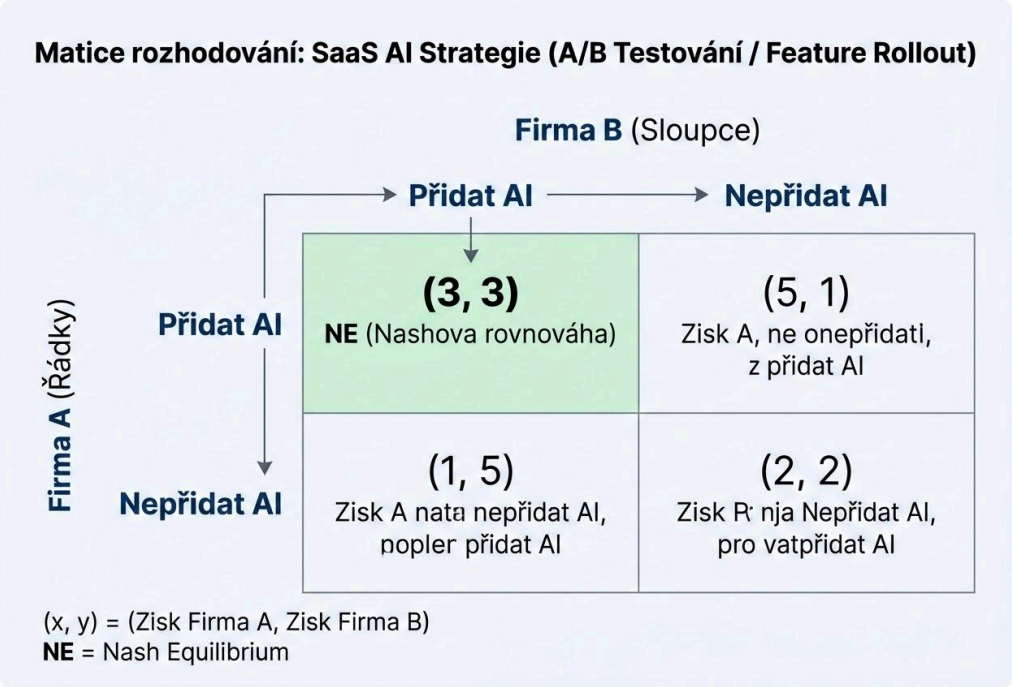
nemění. V Nashově rovnováze každý agent jedná optimálně vzhledem k rozhodnutím ostatních agentů.

- Pokud má každý z agentů dominantní strategii a racionálně ji volí, systém se nachází v Nashově rovnováze.
- Pro jeden problém může existovat více Nashových rovnováh.
- **Typy Nashovy rovnováhy**:
  - *Ryzí*: Každý agent volí ryzí strategii a žádný hráč nemůže jednostranně změnou strategie zlepšit svůj užitek.
  - *Smíšená*: Alespoň jeden agent volí smíšenou strategii a žádný hráč nemůže jednostranně změnou strategie zlepšit svůj užitek.
- **Výpočet**: Pro ryzí strategie se využívá lineární programování, pro smíšené strategie nelineární programování.

Vizualizace: Matice rozhodování v Software Architecture (A/B testing / Feature rollout)
Uvažujme dva konkurenty (Firma A a Firma B), kteří zvažují zavedení AI asistenta do svého editoru code review:

```
None
                     Firma B: Přidat AI       Firma B: Nepřidat AI
Firma A: Přidat AI    [ (3, 3) NE           ] [ (5, 1)             ]
Firma A: Nepřidat AI  [ (1, 5)              ] [ (2, 2)             ]
```

Výplaty jsou uvedeny jako (Zisk A, Zisk B). Stav (3, 3), kdy oba přidají AI, představuje Nashovu rovnováhu (NE), protože ani jedna firma nemá motivaci jednostranně funkci odstranit.



Reflexní otázka k zamyšlení (Kapitola 1)

- *Dokážete identifikovat situaci z vašeho posledního IT projektu nebo týmové spolupráce, která měla charakter Vězňova dilematu? Jaké rozhodnutí by posunulo stav od Nashovy rovnováhy k Paretovu optimu?*

Případová studie / Příklad: Vězňovo dilema

Policie zadržela dva podezřelé – Adama a Boba – a výslech probíhá odděleně.

- Pokud se oba přiznají / budou svědčit proti sobě, dostanou oba trest 6 let.
- Pokud jeden udá druhého, udavač je osvobozen a druhý dostane 10 let.
- Pokud oba odmítnou vypovídat, budou odsouzeni za přestupky na 2 roky.

Dominantní strategie: Pro oba je lákavá vidina osvobození, takže dominantní strategií pro Adama i Boba je mluvit (svědčit).
Nashova rovnováha: Oboustranná volba dominantní strategie vede ke stavu, kdy jsou oba odsouzeni na 6 let.
Software Engineering paralel k Vězňovu dilematu (Refaktoring vs. Technický dluh):
Dva vývojové týmy pracují na společném monorepu:

```
None
                     Tým B: Psát čistý kód     Tým B: Psát „spaghetti“ kód
Tým A: Psát čistý kód [ (5, 5) - Paret. opt. ] [ (1, 8)                  ]
Tým A: „Spaghetti“    [ (8, 1)               ]  [ (2, 2) - Nashova rovn.  ]
```

Oba týmy láká ušetřit čas a psát nekvalitní kód na úkor druhého (zisk 8). Dominantní strategie obou týmů však vede k zaplevelení repozitáře a celkově nejhoršímu udržitelnému stavu (2, 2).

1.4 Paretovo optimum

Paretovo optimum je stav, ve kterém není možné zlepšit užitek jednoho agenta bez současného snížení užitku alespoň jednoho jiného agenta.

- Pro jeden problém může existovat více paretovsky optimálních profilů.
- Paretovo optimum může upřednostňovat některé agenty na úkor jiných.
- Výpočet probíhá pomocí lineárního či nelineárního programování, případně iteračně.

Vizualizace Paretova optima v Product Managementu (Prioritizace funkcí):
Při prioritizaci feature requestů mezi Enterprise klienty a SMB klienty:

```
None
Dostupné vývojářské kapacity: 100 MD
 - Konfigurace A: 80 MD Enterprise, 20 MD SMB (Užitek: Ent=90, SMB=20) ->
Pareto-optimální
 - Konfigurace B: 50 MD Enterprise, 50 MD SMB (Užitek: Ent=60, SMB=60) ->
Pareto-optimální
 - Konfigurace C: 30 MD Enterprise, 30 MD SMB (40 MD nebylo využito)   -> Není
Pareto-optimální
```

Kapitola 2: Teorie her

Praktické cvičení 1: Návrh cenové strategie SaaS platformy

Scénář: Dva konkurující si startupy vyvíjející B2B SaaS nástroj pro automatizaci CI/CD kanálů zvažují, zda zavést plně zdarma dostupnou verzi („Freemium“), nebo zachovat pouze placené plány („Paid-only“).

- Pokud oba zvolí Freemium, rozdělí si trh napůl, ale náklady na bezplatné uživatele sníží zisk obou na 2 mil. Kč.

- Pokud oba zvolí Paid-only, udrží vysoké marže a obě firmy získají zisk 5 mil. Kč.
- Pokud jedna firma zvolí Freemium a druhá Paid-only, firma s Freemium získá většinu trhu (zisk 8 mil. Kč), zatímco druhá firma ztratí zákazníky (zisk 1 mil. Kč).

Úkoly pro samostudium:

1. Sestavte matici rozhodování (payoff matrix) pro tento SaaS scénář.
2. Určete dominantní strategii pro obě firmy.
3. Nalezněte Nashovu rovnováhu a posuďte, zda je zároveň Paretovým optimem.

Výukové cíle kapitoly:

- Definovat základní pojmy teorie her (hra, hráč, tah, racionalita, užitková funkce).
- Klasifikovat hry podle různých kritérií.

2.1 Úvod a základní pojmy

Teorie her je matematický nástroj pro analýzu rozhodovacích situací, kde úspěch jedné strany závisí na rozhodnutích ostatních. Agenta v MAS lze chápat jako „hráče", který maximalizuje svůj užitek.

- **Hra**: Model situace popisující strategickou interakci dvou a více hráčů.
- **Hráč**: Účastník hry.
- **Tah (akce)**: Přípustná varianta chování hráče.
- **Racionalita hráčů**: Základní předpoklad. Očekává se, že hráč si volí cíle na základě stabilních preferencí, volí nejefektivnější tahy a dokáže situace seřadit úplně (pokrývá všechny situace) a tranzitivně ($A > B$ a $B > C \Rightarrow A > C$).
- **Užitková funkce (výplatní / účelová)**: Numerické ohodnocení koncového stavu hry dané preferencemi hráče. Cílem hráče je maximalizace její hodnoty.
- **Aplikace základních pojmů v řízení vývoje produktů:**
    - *Hráči:* Product Lead, Engineering Lead a Security Lead při plánování roadmapy.
    - *Tah:* Alokovat 40 % kapacity na bezpečnostní refaktoring nebo na nové UX rozhraní.
    - *Užitková funkce:* U(Product) = Zvýšení MRR; U(Security) = Snížení počtu CVE zranitelností pod kritickou hranici.

2.2 Klasifikace her

Hry se liší podle:

4. **Počtu hráčů**
5. **Počtu přípustných tahů**:
    - *Konečné*: Všichni hráči mají konečné množství přípustných tahů (naprostá většina reálných problémů).
    - *Nekonečné*: Alespoň jeden hráč má nekonečně mnoho tahů (cílem je udržet se ve hře, např. válka ve Vietnamu).
6. **Míry konkurence**:
    - *S nulovým součtem (kompetitivní)*: Vítěz získává na úkor ostatních (šachy, piškvorky).
    - *S nenulovým součtem*: Zisk jednoho neznamená nutně ztrátu pro ostatní (podnikání, politika).
7. **Způsobu hraní**:
    - *Jednotahové*: Hráči táhnou současně (kámen, nůžky, papír).
    - *Sekvenční*: Hráči se v tazích střídají (šachy, piškvorky, poker).
8. **Dostupnosti informace**:
    - *S úplnou informací* (šachy).
    - *S neúplnou informací* (poker).
9. **Spolupráce**:
    - Nekooperativní, kooperativní, hybridní.
10. **Míry náhody**:
    - *Deterministické*: Bez prvku náhody (šachy, kámen nůžky papír).
11. **Příklady klasifikace ze světa softwaru:**
    - *Aukce reklamních pozic (Google Ads / AdTech):* Jednotahová hra s neúplnou informací (hráči neznají maximální příhozy ostatních).
    - *Válka platforem (App Store vs. Epic Games):* Nekooperativní sekvenční hra s nenulovým součtem a dlouhodobým horizontem.
    - *Load Balancing v distribuovaných systémech:* Stochastická hra, kde požadavky přicházejí dle pravděpodobnostního rozdělení.
    - *Stochastické*: S prvkem náhody (monopoly, poker).

Kapitola 3: Deterministické hry s úplnou informací a algoritmus Minimax

Reflexní otázka k zamyšlení (Kapitola 2)

- *Většina reálných aplikací AI (např. automatické obchodní systémy nebo autonomní řízení) pracuje s neúplnou informací a stochastickým prostředím. Jaké konkrétní výzvy to přináší oproti deterministickým hrám typu Šachy?*

Výukové cíle kapitoly:

- Vysvětlit princip fungování algoritmu Minimax a jeho vlastnosti.
- Popsat princip alfa-beta ořezávání.
- Vysvětlit použití heuristické evaluační funkce a rozhodování v reálném čase.

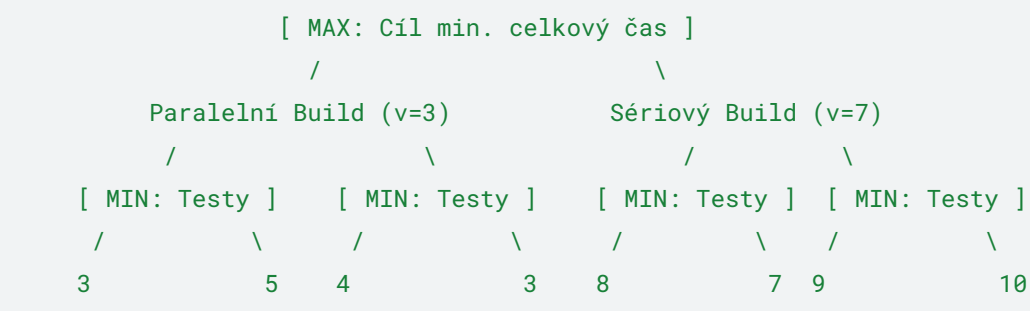
3.1 Algoritmus Minimax

V sekvenčních kompetitivních hrách 2 hráčů (hráč MAX a hráč MIN) hledáme optimální strategii pro hráče MAX, který maximalizuje užitek, zatímco MIN jej minimalizuje.

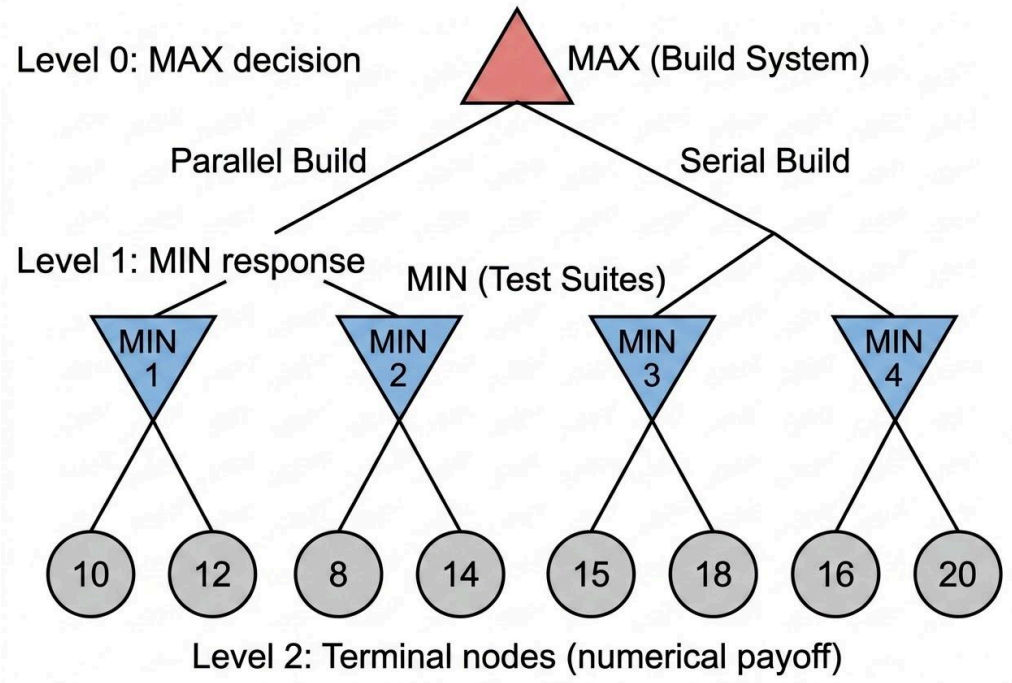
- **Charakteristika Minimaxu**:
  - Vždy prohledává celý strom hry.
  - Podobá se prohledávání do hloubky (DFS).
  - Zaručuje nalezení optimálního řešení.
  - *Důležité*: Nalezení optimálního řešení nezaručuje výhru hráče MAX (výsledkem může být remíza nebo nemožnost vyhrát).
  - **Hlavní nevýhoda**: Vysoká časová náročnost z důvodu procházení celého stromu.

Vizualizace: Minimax strom v optimalizaci CI/CD buildů (Hráč MAX = Build System, Hráč MIN = Zátěž/Sled testů)

None



Hráč MAX volí větve vedoucí k minimální hodnotě pesimistického scénáře, který vybírá protivník (MIN).



3.2 Minimax s alfa-beta ořezáváním

Pro zefektivnění Minimaxu se zavádí alfa-beta ořezávání. Používají se dvě meze:

- **α (alfa)**: Nejlepší doposud nalezená hodnota v prozkoumávané větví z pohledu hráče MAX (nejvyšší hodnota).
- **β (beta)**: Nejlepší doposud nalezená hodnota v prozkoumávané větví z pohledu hráče MIN (nejnižší hodnota).

Větve, které nemohou ovlivnit konečné rozhodnutí, jsou oříznuty.

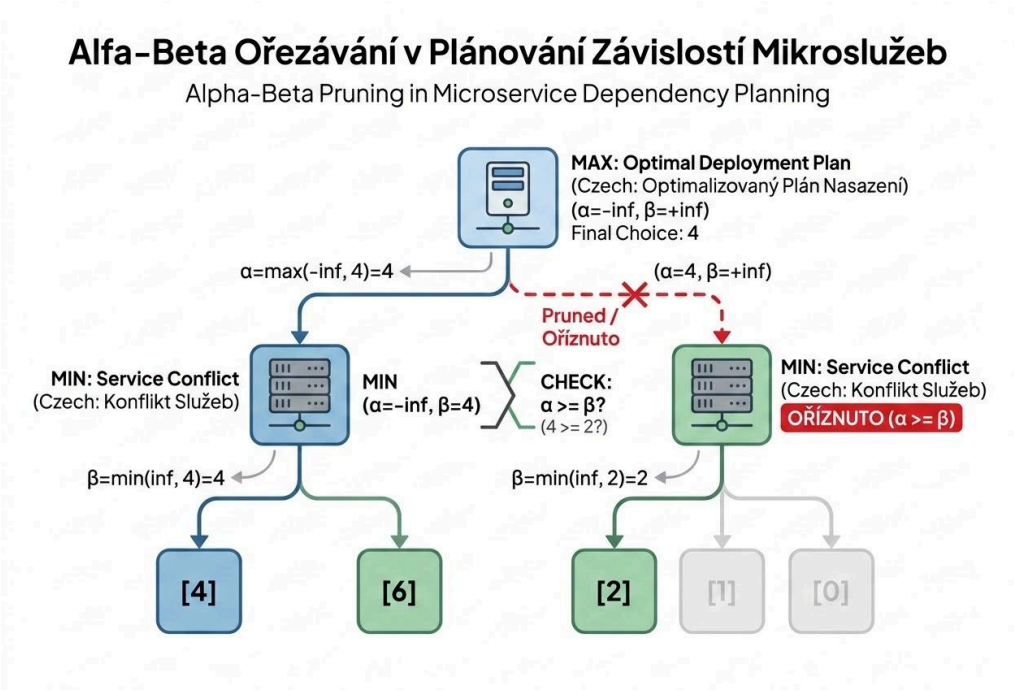
- Paměťová náročnost zůstává stejná jako u původního Minimaxu.
- Časová náročnost výrazně závisí na pořadí ohodnocování stavů.

Vizualizace: Alfa-beta ořezávání v plánování závislostí mikroslužeb

```
None
                [ MAX (alfa=-inf, beta=+inf) ]
                 /                           \
```

```
     [ MIN (alfa=-inf, beta=4) ]       [ MIN (alfa=4, beta=2) ] -> OŘÍZNUTO (alfa
>= beta)
       /                   \             /
    [4]                     [6]       [2]  (Další sousední větve s hodnotami 1, 0
se již neprochází!)
```

3.3 Rozhodování v reálném čase a heuristická evaluační funkce

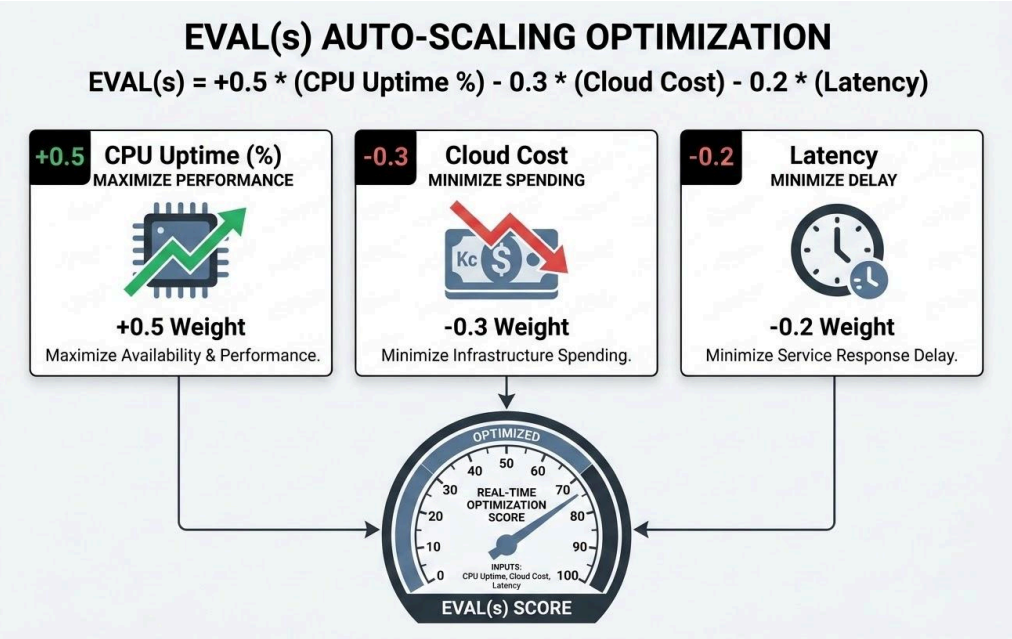


V reálných aplikacích existuje časový limit a nelze procházet strom až do cílových koncových stavů.

- **Minimax s omezenou hloubkou (Depth-limited Minimax)**: Prohledávání stromu se zastaví v limitní hloubce d_max pomocí funkce CUTTING-OFF(s,d).
- **Nastavení limitu d_max**:
  - *Fixní hodnota*: Zvolena dle časového limitu.
  - *Dynamická hodnota*: Využívá iterativní prohledávání do hloubky, zastaví se po vypršení času a použije největší kompletně ohodnocenou hloubku.
- **Heuristická evaluační funkce (EVAL(s))**: Slouží k odhadu užitku nekoncových stavů v limitní hloubce. Bývá definována jako vážený součet charakteristik: EVAL(s) = w1 * f1(s) + w2 * f2(s) + ... + wn * fn(s).

- *Monotónní transformace*: Přesné hodnoty vrácené funkcí EVAL(s) nejsou kritické, chování algoritmu zůstává zachováno pro libovolnou monotónní transformaci.
- **Příklad heuristiky v řízení vývoje (EVAL funkce pro údržbu kódové báze):**

```
None
EVAL(System_State) = w1 * (Code Coverage %) - w2 * (Kritické zranitelnosti) - w3 * (Průměrná doba buildu)
Kde váhy např. w1 = 0.3, w2 = 10.0, w3 = 0.5 vyjadřují prioritu jednotlivých metrik.
```



Praktické cvičení 2: Optimalizace AI agenta v cloudové infrastruktuře

Scénář: Navrhujete AI agenta (MAX), který spravuje auto-scaling v cloudu. Protivníkem (MIN) je nepředvídatelná uživatelská zátěž a DDoS útoky. Agent má k dispozici heuristickou funkci: EVAL(State) = 0.5 * Uptime% - 0.3 * Cost - 0.2 * Latency_ms.

Úkoly pro samostudium:

12. Vysvětlete, proč je nutné použít Minimax s omezenou hloubkou (Depth-limited), pokud musí agent reagovat v reálném čase do 100 ms.
13. Jak by alfa-beta ořezávání pomohlo snížit výpočetní nároky při prohledávání stromu možných konfigurací serverů?

Slovník klíčových pojmů

Pojem	Definice / Význam
Multiagentní systém (MAS)	Systém složený z více autonomních agentů, kteří navzájem interaktivně působí v rámci společného prostředí.
Nashova rovnováha	Stav, ve kterém žádný hráč nemůže jednostrannou změnou strategie zvýšit svůj užitek.
Paretovo optimum	Stav, kdy nelze zvýšit užitek jednoho hráče, aniž by se snížil užitek alespoň jednoho z ostatních.
Dominantní strategie	Strategie, která poskytuje nejvyšší výnos bez ohledu na volby ostatních hráčů.
Minimax	Prohledávací algoritmus pro deterministické hry s úplnou informací, vybraný k minimalizaci maximální možné ztráty.
Alfa-Beta ořezávání	Optimalizační technika pro algoritmus Minimax, která eliminováním nerelevantních větví zrychluje prohledávání.

Doporučená literatura a studijní zdroje

RUSSELL, Stuart J. a Peter NORVIG. Artificial Intelligence: A Modern Approach. 4th ed. Pearson, 2020. ISBN 978-0134610993. (Kapitoly 5: Adversarial Search a 17: Multi-Agent Decision Making).

SHOAM, Yoav a Kevin LEYTON-BROWN. Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations. Cambridge University Press, 2008. ISBN 978-0521899437.

[Volitelně] MYERSON, Roger B. Game Theory: Analysis of Conflict. Harvard University Press, 1997. ISBN 978-0674341166.